บทพูดนำเสนออัปเดตความคืบหน้า

ระบบจัดการคำสั่งซื้อและตรวจสอบคลังสินค้า ร้านวัสดุก่อสร้างอำพรคอนกรีต (อำพร OMS)

ใช้คู่กับสไลด์ Canva 25 หน้า · อ่านซ้อมก่อนเข้าพบอาจารย์ที่ปรึกษา

วิธีใช้เอกสารนี้

- แต่ละหน้าแบ่งเป็น **พูด** (สิ่งที่พูดออกไป) **ถ้าอาจารย์ถาม** (คำถามที่น่าจะโดน พร้อมคำตอบสั้น ๆ) และ **ดูในเล่ม** (ที่อยู่ของข้อมูลจริง ไว้เปิดเช็กก่อนพูด)
- ไม่ต้องท่องคำต่อคำ จำแค่ประเด็นแล้วพูดด้วยคำของตัวเอง
- ตัวเลขทุกตัวมาจากเล่ม ถ้าจำไม่ได้ให้บอกว่า “อยู่ในตารางที่ ... ครับ” แล้วเปิดให้ดู ดีกว่าเดา
- บทพูดชุดเดียวกันนี้อยู่ในโน้ตผู้นำเสนอ (Speaker Notes) ของแต่ละสไลด์ใน Canva แล้ว

หน้า 1 · อำพร OMS อัปเดตความคืบหน้า

พูด

สวัสดีครับอาจารย์ วันนี้ผมมาอัปเดตโครงงานอำพร OMS ครับ เป็นระบบขายหน้าร้านกับคลังสินค้าของร้านวัสดุก่อสร้างอำพรคอนกรีต ผมจะเล่าตามลำดับเลยครับ เริ่มจากขอบเขต แล้วไล่ไปว่าได้ requirement อะไร แบ่งเป็น use case ยังไง ออกแบบ C2 C3 ยังไง แล้วจบที่ว่าแต่ละ use case วิ่งผ่านโค้ดส่วนไหนบ้าง กับผลทดสอบครับ

หน้า 2 · ขอบเขตของโครงงาน

พูด

ขอบเขตมี 5 ข้อครับ อยู่ในหัวข้อ 1.3

ข้อแรกเป็นเว็บที่ใช้ได้ทั้งคอมและมือถือ มีผู้ใช้ 3 บทบาท คือพนักงานขาย พนักงานคลัง แล้วก็ผู้จัดการ

ข้อสองคือเข้าสู่ระบบ แยกสิทธิ์ตามบทบาท และจัดการสินค้า

ข้อสามคือหน้าขาย มี QR พร้อมเพย์ พิมพ์ใบเสร็จ แล้วส่งใบสั่งเข้าคิวคลังทันที

ข้อสี่คือคิวคลังแบบเรียลไทม์ กดยืนยันจ่ายแล้วตัดสต็อกทีเดียวจบ

ข้อห้าคือแดชบอร์ด ประวัติการทำรายการ กับแจ้งเตือนในแอป

ส่วนที่ไม่ทำ เช่น ระบบบัญชีเต็มรูปแบบ ตรวจยอดโอนกับธนาคารอัตโนมัติ แอปมือถือแบบ native กับแจ้งเตือนผ่าน LINE ครับ

ถ้าอาจารย์ถาม ทำไมไม่ตรวจยอดโอนอัตโนมัติ

ต้องใช้ API ของธนาคาร ซึ่งร้านต้องไปสมัครกับธนาคารเอง และเกินเวลาของโครงงานครับ ตอนนี้พนักงานดูยอดเข้าในแอปธนาคารแล้วค่อยกดยืนยันในระบบ

ดูในเล่ม หัวข้อ 1.3 ขอบเขต และ 1.3 ส่วนนอกขอบเขต

หน้า 3 · จากขอบเขตเป็น Requirement 18 ข้อ

พูด

จากขอบเขต 5 ข้อ ผมได้ requirement ทั้งหมด 18 ข้อครับ เป็น FR 9 ข้อ กับ NFR 9 ข้อ
หน้านี้โชว์ว่าแต่ละข้อมาจากขอบเขตข้อไหน เช่น ขอบเขตข้อ 1.3.4 คิวคลังกับตัดสต็อก ได้ FR-04 FR-05 FR-06 แล้วก็ NFR-02 เรื่องคิวต้องขึ้นภายใน 3 วินาที กับ NFR-04 เรื่องกดพร้อมกันแล้วต้องไม่พัง
มี 2 ข้อที่ไม่ได้มาจากขอบเขตตรง ๆ ครับ คือ NFR-08 ที่มาจากสมมติฐานว่าใช้สถาปัตยกรรม 3 ชั้น กับ NFR-09 ที่ระบบต้องติดตั้งที่ร้านได้จริง

ถ้าอาจารย์ถาม ทำไมต้องมี NFR-09
เพราะถ้าติดตั้งที่ร้านไม่ได้ ระบบก็ไม่ได้ใช้จริงครับ ตอนนี้ผู้จัดการร้านลงเองตามคู่มือ ใช้ประมาณ 5 นาที ผ่านแล้วครับ

ดูในเล่ม บทที่ 3 ตาราง FR และ NFR · หน้าสืบย้อน Traceability

หน้า 4 · Requirement มาจาก Stakeholder ไหน

พูด

Requirement ไม่ได้คิดเองครับ ผมไปดูการทำงานที่ร้าน แล้วคุยกับเจ้าของร้านกับพนักงาน สิ่งที่เจอผมจดเป็นหลักฐาน EV-01 ถึง EV-09
Stakeholder มี 7 กลุ่มครับ เป็นคน 6 กลุ่ม คือพนักงานขาย พนักงานคลัง ผู้จัดการ ลูกค้า เจ้าของร้าน กับผู้จัดทำ แล้วก็ระบบอัตโนมัติอีก 1
ยกตัวอย่างนะครับ
พนักงานขายตอนนี้เขียนบิลกับคิดเงินเอง คือ EV-01 เลยได้ FR-03
ลูกค้าต้องถือบิลเดินไปคลังแล้วยืนรอ คือ EV-04 เลยได้ FR-04 กับ NFR-02 ให้คลังเห็นใบสั่งทันที
ผู้จัดการอยากรู้ว่าของใกล้หมดเมื่อไร อยากดูย้อนหลังว่าใครทำอะไร เลยได้ FR-07 FR-08 FR-09
ส่วนผู้จัดทำ เป็นคนที่ต้องดูแลกับติดตั้งระบบต่อ เลยเป็นเจ้าของ NFR-08 เรื่องแยกชั้นให้แก้โค้ดง่าย กับ NFR-09 เรื่องติดตั้งที่ร้านได้

ถ้าอาจารย์ถาม ทำไมไม่นับอาจารย์ที่ปรึกษาเป็น stakeholder
อาจารย์ตรวจโครงงานครับ แต่ไม่ได้ใช้ระบบ และไม่ได้อะไรจากระบบ ผมเลยนับเฉพาะคนที่ได้หรือเสียประโยชน์จากตัวระบบจริง ๆ

ถ้าอาจารย์ถาม แล้วทำไมผู้จัดทำนับ

เพราะหลังส่งมอบ ผู้จัดทำต้องแก้โค้ดและลงเครื่องใหม่ให้ร้าน ถ้าโค้ดแยกชั้นไม่ดี หรือติดตั้งยาก คนที่ลำบากคือผู้จัดทำครับ

ถ้าอาจารย์ถาม เก็บความต้องการมายังไง

ไปสังเกตการทำงานที่ร้าน กับสัมภาษณ์เจ้าของร้านและพนักงาน จดสิ่งที่เห็นเป็น EV ทีละข้อ แล้วค่อยแปลงเป็น requirement ครับ

ดูในเล่ม บทที่ 3 ตารางหลักฐาน EV · เอกสารผู้มีส่วนได้ส่วนเสีย หัวข้อ 1

หน้า 5 · ประเภทของ Requirement

พูด

สรุปตัวเลขครับ requirement 18 ข้อ มาจาก stakeholder 7 กลุ่มที่เพิ่งพูดไป

แบ่งเป็นสองแบบครับ

FR 9 ข้อ คือสิ่งที่ระบบต้องทำได้ เช่น เข้าสู่ระบบ สร้างคำสั่งซื้อ ตัดสต็อก แจ้งเตือน

NFR 9 ข้อ คือระบบต้องดีแค่ไหน ผมแยกตามด้านคุณภาพครับ

ด้านความเร็วมี 3 ข้อ คิวต้องขึ้นภายใน 3 วินาที ใบเสร็จออกภายใน 5 วินาที แล้วรับผู้ใช้พร้อมกัน 10 เซสชัน

ด้านความปลอดภัยคือ NFR-01 ตรวจสิทธิ์ทุกคำขอ ด้านความน่าเชื่อถือคือ NFR-04 กดพร้อมกันแล้วสต็อกไม่เพี้ยน

ที่เหลือคือ ใช้บนมือถือได้ ใช้ได้หลายเบราว์เซอร์ แยกชั้นให้แก้โค้ดง่าย กับติดตั้งที่ร้านได้ครับ

ถ้าอาจารย์ถาม FR กับ NFR ต่างกันยังไง

FR บอกว่าระบบทำอะไรครับ เช่น ตัดสต็อกได้ ส่วน NFR บอกว่าทำได้ดีแค่ไหน เช่น ตัดแล้วคิวขึ้นภายใน 3 วินาที กดพร้อมกันแล้วไม่พัง NFR ทุกข้อผมใส่เกณฑ์ที่วัดได้ไว้ เลยทดสอบได้ว่าผ่านหรือไม่

ดูในเล่ม บทที่ 3 ตาราง FR และ NFR

หน้า 6 · Requirement แบ่งเป็น Use Case 11 ตัว

พูด

FR 9 ข้อ ผมแตกออกมาเป็น use case 11 ตัวครับ อยู่ตารางที่ 3.8

ส่วนใหญ่ 1 FR ต่อ 1 UC แต่บางข้อแตกเป็น 2 ครับ เช่น FR-03 แตกเป็น UC-03 กับ UC-04 เพราะคนทำคนละคน

UC-03 พนักงานขายเป็นคนสร้างคำสั่งซื้อ ส่วน UC-04 ระบบเป็นคนออกเลขที่เอง

กลับกัน FR-05 กับ FR-06 รวมเป็น UC-07 ตัวเดียว เพราะยืนยันจ่ายกับตัดสต็อกต้องเกิดพร้อมกันในครั้งเดียว แยกไม่ได้ครับ

ถ้าอาจารย์ถาม ใช้หลักอะไรแบ่ง use case
ดูว่าใครเป็นคนทำ กับงานนั้นจบในตัวเองไหมครับ ถ้าคนทำต่างกันก็แยก ถ้าต้องเกิดพร้อมกันเสมอก็รวม

ดูในเล่ม ตารางที่ 3.8

หน้า 7 · Use Case Diagram รวมทั้งระบบ

พูด
นี่คือ use case diagram ทั้งระบบครับ มี UC-01 ถึง UC-11 กับผู้ใช้ 3 บทบาท
ทุกคนต้องผ่าน UC-01 เข้าสู่ระบบก่อน พนักงานขายใช้ UC-03 พนักงานคลังใช้ UC-05 ถึง UC-07 ผู้จัดการดูแลสินค้า แดชบอร์ด ประวัติ กับแจ้งเตือน
ส่วน UC-04 กับการสร้างแจ้งเตือนใน UC-11 ระบบทำเองครับ

ดูในเล่ม SRS ภาคผนวก ก รูปที่ 2 · บทที่ 3 แผนภาพแยกตามกลุ่มงาน

หน้า 8 · สถาปัตยกรรมระดับ C2

พูด
ต่อไปเป็นการออกแบบครับ ระดับ C2 ผมแบ่งเป็น 3 ชั้น คือหน้าจอ ตรรกะ แล้วก็ข้อมูล แต่ติดตั้งจริงเป็น 4 ตัว หน้าเว็บใช้ React หลังบ้านใช้ Node.js กับ Express ฐานข้อมูลใช้ MySQL แล้วมีบริการพิมพ์ใบเสร็จแยกออกมาอีกตัว
ที่แยกเพราะบริการพิมพ์ต้องรันบนเครื่องที่ต่อเครื่องพิมพ์อยู่ครับ ถึงจะอยู่ชั้นตรรกะเหมือนหลังบ้าน แต่ต้องติดตั้งแยก
หน้าจอคุยกับหลังบ้านสองแบบครับ คำขอทั่วไปใช้ REST ส่วนคิวเรียลไทม์ใช้ WebSocket และมี SSE เป็นทางสำรอง

ถ้าอาจารย์ถาม ทำไมคิวไม่ใช้วิธีรีเฟรชทุก ๆ กี่วินาที
ถ้ารีเฟรชเรื่อย ๆ จะเปลืองและยังช้ากว่า เพราะต้องรอรอบถัดไป WebSocket ให้หลังบ้านผลักข้อมูลมาทันทีที่มีใบสั่งใหม่ ผลที่วัดได้คือช้าสุด 80 มิลลิวินาทีครับ

ดูในเล่ม ตารางที่ 3.17 · รูปที่ 3.7

หน้า 9 · C2 Container Diagram

พูด

นี่คือภาพ C2 ครับ รูปที่ 3.7 ในเล่ม ผู้ใช้ 3 บทบาทเข้ามาทางเว็บ เว็บคุยกับ Backend API หลังบ้านเก็บข้อมูลใน MySQL แล้วสั่งพิมพ์ผ่าน Print Service ไปที่เครื่องพิมพ์ใบเสร็จ QR พร้อมเพย์ใช้ไลบรารีภายนอกสร้างครับ

หน้า 10 · ส่วนประกอบระดับ C3

พูด

ซูมเข้าไปในหลังบ้าน ระดับ C3 มี 10 โมดูล กับส่วนสนับสนุนอีก 3 ตัวครับ ตารางที่ 3.13 กับ 3.14 โมดูลแบ่งตามงาน เช่น Auth ดูเรื่องเข้าสู่ระบบ Order ดูเรื่องสร้างคำสั่งซื้อ Stock Deduction ดูเรื่องตัดสต็อก ที่ผมแยก Stock Deduction ออกจาก Dispatch เพราะอยากเรียกมันตรง ๆ พร้อมกันหลาย ๆ ครั้งตอนทดสอบได้ครับ เพื่อพิสูจน์ NFR-04 ว่ากดพร้อมกันแล้วสต็อกไม่เพี้ยน

หน้า 11 · C3 Component Diagram

พูด

นี่คือภาพ C3 ครับ รูปที่ 3.8 ทุกคำขอจะผ่าน Auth Middleware ก่อนเข้าโมดูล ชื่อโมดูลในภาพตรงกับโฟลเดอร์ในโค้ดเลยครับ อยู่ใน backend/modules

ถ้าอาจารย์อยากดู ผมเปิดโฟลเดอร์ให้ดูได้เลยครับ

หน้า 12 · UC-01 เข้าสู่ระบบและควบคุมสิทธิ์

พูด

UC-01 เป็นด่านแรกของทุกคนครับ พนักงานกรอกชื่อกับรหัสที่หน้า login ส่งไป POST /api/auth/login โมดูล Auth เทียบรหัสกับตาราง users

ถ้าถูก ระบบให้บัตรผ่านหรือโทเคน ที่บอกว่าคนนี้เป็นบทบาทอะไร แล้วจดลงประวัติว่าใครเข้าระบบตอนไหน

หลังจากนั้นทุกคำขอต้องผ่าน Auth Middleware ไม่มีบัตรตอบ 401 มีบัตรแต่ไม่ใช่หน้าที่ตัวเองตอบ 403

ทดสอบด้วย TC-01 ลองไม่มีบัตร บัตรปลอม บัตรของบทบาทอื่น ถูกปฏิเสธครบทุกกรณี ผ่านครับ

อธิบายเส้น

ก่อนเข้า use case แรก ขออธิบายวิธีอ่านภาพหน่อยครับ

อ่านจากซ้ายไปขวา 5 ช่อง คือ use case หน้าจอ API โมดูลหลังบ้าน แล้วก็ตารางในฐานข้อมูล

เส้นสีส้มคือทางที่ use case นี้วิ่งผ่านจริง เส้นที่ข้ามจากช่องหนึ่งไปอีกช่องคือเรียกต่อกัน
เส้นโค้งที่ออกทางขวาของช่อง Backend แล้ววนกลับเข้าช่องเดิม คือโมดูลเรียกโมดูลด้วยกัน
กล่องขอบประ คือของที่อยู่นอกโค้ดเรา เช่น ไลบรารี QR พร้อมเพย์ กับโปรแกรมพิมพ์ใบเสร็จ
ทุกเส้นผมไล่มาจากโค้ดจริง หน้าจอดูจากโฟลเดอร์ pages API ดูจาก app.js โมดูลดูจากคำสั่ง require ในแต่ละโมดูลครับ
เส้นแรก จาก UC-01 ไปหน้าจอ login คือ use case นี้ทำที่หน้าจอนี้ครับ
จากหน้าจอ login มี 2 เส้น ไป POST /api/auth/login กับ GET /api/auth/me คือหน้าจอนี้ส่งคำขอไปหลังบ้านผ่าน API สองตัวนี้
ทั้ง 2 เส้นวิ่งเข้าโมดูล Auth เหมือนกันครับ
เส้นโค้งทางขวาของช่อง Backend คือโมดูลเรียกกันเองครับ Auth เรียก Audit Log
เส้นสุดท้ายไปช่องขวาสุด คือโมดูลไหนใช้ตารางไหนครับ Auth ใช้ตาราง users · Audit Log ใช้ตาราง audit_logs

ถ้าอาจารย์ถาม รหัสผ่านเก็บยังไง
เก็บแบบเข้ารหัสด้วย bcrypt ครับ ไม่ได้เก็บตัวจริง ตรงกับ NFR-01

หน้า 13 · UC-02 จัดการสินค้าและจุดสั่งซื้อเพิ่ม

พูด

UC-02 ผู้จัดการดูแลสินค้าครับ เพิ่ม แก้ราคา ปรับยอด ที่หน้า products
คนอื่นดูได้อย่างเดียว แก้ไม่ได้ เพราะเส้นทางเขียนกำหนดไว้ว่าเฉพาะผู้จัดการ
ปรับยอดทีไร ระบบจด stock_movements ว่าเปลี่ยนไปเท่าไร เหลือเท่าไร กับจดประวัติใน audit_logs พร้อมกัน
แล้วเช็กว่าต่ำกว่าจุดสั่งซื้อเพิ่มไหม ถ้าต่ำก็แจ้งเตือน
ทดสอบด้วย TC-02 แก้จุดสั่งซื้อเพิ่มแล้วค่าที่บันทึกตรงทุกครั้ง มีประวัติว่าใครแก้ ผ่านครับ

อธิบายเส้น

เส้นแรก จาก UC-02 ไปหน้าจอ products คือ use case นี้ทำที่หน้าจอนี้ครับ
จากหน้าจอ products มีเส้นเดียว ไป /api/products (GET POST PUT PATCH) คือหน้าจอนี้ส่งคำขอไปหลังบ้านผ่าน API ตัวนี้
/api/products (GET POST PUT PATCH) วิ่งเข้าโมดูล Product
เส้นโค้งทางขวาของช่อง Backend คือโมดูลเรียกกันเองครับ Product เรียก Alert Engine กับ Audit Log
เส้นสุดท้ายไปช่องขวาสุด คือโมดูลไหนใช้ตารางไหนครับ Product ใช้ตาราง products กับ stock_movements · Alert Engine ใช้ตาราง notifications · Audit Log ใช้ตาราง audit_logs

---

หน้า 14 · UC-03 สร้างคำสั่งซื้อและออกใบเสร็จ

พูด

UC-03 คือหน้าร้านครับ พนักงานขายเลือกสินค้าลงตะกร้าที่หน้า pos
ระบบคิดเงินจากราคาในฐานข้อมูลเอง ไม่เชื่อราคาที่หน้าจอส่งมา กันคนแก้ราคาเอง แล้วสร้าง QR พร้อมเพย์ตามยอด
ลูกค้าสแกนจ่ายแล้ว พนักงานกดยืนยัน ระบบบันทึกใบสั่งกับรายการสินค้าพร้อมกันทีเดียว
จากนั้นสั่งพิมพ์ใบเสร็จ และส่งใบสั่งไปโผล่ที่จอคิวของคลังทันที สองอย่างนี้เกิดจากการกดครั้งเดียวกันตาม BR-01
ถ้าเครื่องพิมพ์เสีย ใบสั่งก็ยังเข้าคิวปกติ แค่ขึ้นเตือนว่าพิมพ์ไม่สำเร็จ (AC-05)
ทดสอบสองข้อครับ TC-03 ยอดกับ QR ตรงทุกครั้ง และ TC-12 พิมพ์ใบเสร็จภาษาไทยกับเครื่องจริงได้ ใช้ประมาณ 2.3 วินาที เกณฑ์คือ 5 วินาที ผ่านครับ

อธิบายเส้น

เส้นแรก จาก UC-03 ไปหน้าจอ pos คือ use case นี้ทำที่หน้าจอนี้ครับ
จากหน้าจอ pos มี 2 เส้น ไป POST /api/orders/quote กับ POST /api/orders คือหน้าจอนี้ส่งคำขอไปหลังบ้านผ่าน API สองตัวนี้
ทั้ง 2 เส้นวิ่งเข้าโมดูล Order เหมือนกันครับ
เส้นโค้งทางขวาของช่อง Backend คือโมดูลเรียกกันเองครับ Order เรียก PromptPay QR Print Client Queue Manager กับ Audit Log
เส้นสุดท้ายไปช่องขวาสุด คือโมดูลไหนใช้ตารางไหนครับ Order ใช้ตาราง orders order_items กับ products · Print Client ส่งงานพิมพ์ไป Print Service ที่เครื่องร้าน · Audit Log ใช้ตาราง audit_logs

ถ้าอาจารย์ถาม ทำไมคิดเงินที่หลังบ้าน ไม่คิดที่หน้าจอ
หน้าจออยู่ในเครื่องผู้ใช้ แก้ได้ครับ ถ้าเชื่อราคาจากหน้าจอ ใครแก้ราคาในเบราว์เซอร์ก็ขายถูกได้ เลยให้หลังบ้านดึงราคาจากตาราง products เองเสมอ อยู่ในฟังก์ชัน priceItems ของโมดูล Order ครับ

หน้า 15 · UC-04 สร้างเลขที่คำสั่งซื้อไม่ซ้ำ

พูด

UC-04 ไม่มีหน้าจอของตัวเองครับ ระบบทำให้ตอนกดยืนยันใน UC-03
ระบบดูเลขล่าสุดของวันแล้วจองไว้ด้วย FOR UPDATE ถ้าอีกเครื่องกดพร้อมกันต้องรอคิว เลยได้เลขถัดไปที่ไม่ซ้ำ รูปแบบคือ ORD วันที่ แล้วตามด้วยลำดับ และฐานข้อมูลยังตั้งไว้ว่าห้ามเลขซ้ำอีกชั้นหนึ่ง
ทดสอบด้วย TC-04 สร้างต่อกัน 100 ใบ กับพร้อมกัน 20 ใบจากสองบัญชี เลขไม่ซ้ำเลยทั้ง 120 ใบ ผ่านครับ

อธิบายเส้น

เส้นแรก จาก UC-04 ไปหน้าจอ pos เพราะ UC-04 ไม่มีหน้าจอของตัวเอง มันเกิดตอนกดยืนยันขายในหน้า pos ครับ

จากหน้าจอ pos มีเส้นเดียว ไป POST /api/orders คือหน้าจอนี้ส่งคำขอไปหลังบ้านผ่าน API ตัวนี้

POST /api/orders วิ่งเข้าโมดูล Order

เส้นสุดท้ายไปช่องขวาสุด คือโมดูลไหนใช้ตารางไหนครับ Order ใช้ตาราง orders

---

หน้า 16 · UC-05 ดูคิวรอจ่ายแบบเรียลไทม์

พูด

UC-05 คลังเห็นใบสั่งใหม่ทันทีครับ เปิดหน้า queue ระบบดึงใบที่รอจ่ายมาแสดง แล้วต่อสาย WebSocket ค้างไว้

พอหน้าร้านขายเสร็จ Queue Manager ก็ผลักใบสั่งใหม่มาที่จอเอง ไม่ต้องกดรีเฟรช

ทดสอบด้วย TC-05 จับเวลาตั้งแต่ขายเสร็จจนขึ้นจอคิว 20 รอบ ช้าสุด 80 มิลลิวินาที ทางสำรองช้าสุด 43 มิลลิวินาที เกณฑ์ 3 วินาที ผ่านครับ

อธิบายเส้น

เส้นแรก จาก UC-05 ไปหน้าจอ queue คือ use case นี้ทำที่หน้าจอนี้ครับ

จากหน้าจอ queue มี 2 เส้น ไป GET /api/queue กับ WebSocket /ws (สำรองเป็น SSE) คือหน้าจอนี้ส่งคำขอไปหลังบ้านผ่าน API สองตัวนี้

ทั้ง 2 เส้นวิ่งเข้าโมดูล Queue Manager เหมือนกันครับ

เส้นสุดท้ายไปช่องขวาสุด คือโมดูลไหนใช้ตารางไหนครับ Queue Manager ใช้ตาราง orders order_items กับ products

---

หน้า 17 · UC-06 ปรับสถานะการจัดของ

พูด

UC-06 คลังกดบอกว่าใบไหนกำลังจัดอยู่ครับ คนอื่นจะได้ไม่จัดซ้ำ

ระบบยอมให้เปลี่ยนสถานะตามลำดับเท่านั้น ข้ามขั้นไม่ได้ แล้วจดประวัติ และทุกจอเห็นสถานะใหม่พร้อมกัน

เรื่องทดสอบ ขอพูดตรง ๆ ครับ UC-06 ยังไม่มีกรณีทดสอบระดับระบบแยกของตัวเอง ในตารางสืบย้อนบทที่ 4 ใช้ TC-05 ร่วมกับ UC-05 เพราะอยู่ใต้ FR-04 เดียวกัน

อธิบายเส้น

เส้นแรก จาก UC-06 ไปหน้าจอ queue คือ use case นี้ทำที่หน้าจอนี้ครับ

จากหน้าจอ queue มีเส้นเดียว ไป PATCH /api/orders/:id/status คือหน้าจอนี้ส่งคำขอไปหลังบ้านผ่าน API ตัวนี้ PATCH /api/orders/:id/status วิ่งเข้าโมดูล Queue Manager
เส้นโค้งทางขวาของช่อง Backend คือโมดูลเรียกกันเองครับ Queue Manager เรียก Audit Log
เส้นสุดท้ายไปช่องขวาสุด คือโมดูลไหนใช้ตารางไหนครับ Queue Manager ใช้ตาราง orders · Audit Log ใช้ตาราง audit_logs

ถ้าอาจารย์ถาม แล้วจะเพิ่ม TC ให้ UC-06 ไหม
ถ้าอาจารย์เห็นว่าควรเพิ่ม ผมจะเพิ่มกรณีลองข้ามสถานะแล้วต้องถูกปฏิเสธครับ

หน้า 18 · UC-07 ยืนยันจ่ายสินค้าและตัดสต็อก

พูด
UC-07 คือหัวใจของระบบครับ ของออกจากร้านเมื่อไร สต็อกลดทันที และไม่มีทางจ่ายซ้ำหรือติดลบ
พนักงานคลังเปิดใบสั่งดูก่อน แล้วกดยืนยันจ่าย ระบบเริ่มทำแบบทำครบทุกขั้นหรือไม่ทำเลย
ขั้นแรกเช็กว่าใบนี้ยังไม่มีใครจ่าย ถ้ามีคนจ่ายไปแล้วปฏิเสธ
ขั้นต่อมาหักสต็อกทีละรายการ ถ้ารายการไหนของไม่พอ ยกเลิกทั้งใบ ไม่มีหักไปครึ่งเดียว
แล้วจดความเคลื่อนไหวกับประวัติ เสร็จแล้วบอกทุกจอ กับเช็กของใกล้หมด
ทดสอบสองข้อครับ TC-06 กดจ่ายใบเดียวกันพร้อมกันสองคำขอ สำเร็จแค่คำขอเดียว สต็อกหักครั้งเดียว และ TC-07 จ่ายเกินของที่มี ยกเลิกทั้งใบ ยอดที่หักไปแล้วกลับค่าเดิม ผ่านทั้งสองข้อครับ

อธิบายเส้น
เส้นแรก จาก UC-07 ไปหน้าจอ dispatch คือ use case นี้ทำที่หน้าจอนี้ครับ
จากหน้าจอ dispatch มี 2 เส้น ไป GET /api/orders/:id กับ POST /api/orders/:id/dispatch คือหน้าจอนี้ส่งคำขอไปหลังบ้านผ่าน API สองตัวนี้
GET /api/orders/:id วิ่งเข้าโมดูล Order · POST /api/orders/:id/dispatch วิ่งเข้าโมดูล Dispatch
เส้นโค้งทางขวาของช่อง Backend คือโมดูลเรียกกันเองครับ Dispatch เรียก Stock Deduction Queue Manager กับ Alert Engine · Stock Deduction เรียก Audit Log
เส้นสุดท้ายไปช่องขวาสุด คือโมดูลไหนใช้ตารางไหนครับ Stock Deduction ใช้ตาราง orders order_items products กับ stock_movements · Audit Log ใช้ตาราง audit_logs · Alert Engine ใช้ตาราง notifications

ถ้าอาจารย์ถาม ถ้าไฟดับตอนกำลังตัดสต็อกจะเป็นยังไง
ธุรกรรมยังไม่ถูก COMMIT ครับ MySQL จะย้อนกลับเอง ข้อมูลไม่ค้างครึ่งทาง

ดูในเล่ม หัวข้อ 3.7.3 · รูปที่ 3.11 · ภาคผนวก ข.3

หน้า 19 · UC-08 แดชบอร์ดปฏิบัติการ

พูด

UC-08 เป็นภาพรวมของวันนี้ครับ ขายไปเท่าไร คิวค้างกี่ใบ อะไรใกล้หมด

Dashboard Aggregator รวมตัวเลขจากใบสั่งกับสินค้าให้ แล้วหน้าจอฟังเหตุการณ์จาก WebSocket พอมีการขายหรือจ่ายของใหม่ตัวเลขก็อัปเดตเอง

ทดสอบด้วย TC-08 เทียบตัวเลขบนแดชบอร์ดกับที่คำนวณจากฐานข้อมูลตรง ๆ ตรงกันทุกตัว ผ่านครับ

อธิบายเส้น

เส้นแรก จาก UC-08 ไปหน้าจอ dashboard-ops คือ use case นี้ทำที่หน้าจอนี้ครับ

จากหน้าจอ dashboard-ops มี 2 เส้น ไป GET /api/dashboard/ops กับ WebSocket /ws (สำรองเป็น SSE) คือหน้าจอนี้ส่งคำขอไปหลังบ้านผ่าน API สองตัวนี้

GET /api/dashboard/ops วิ่งเข้าโมดูล Dashboard Aggregator

เส้นสุดท้ายไปช่องขวาสุด คือโมดูลไหนใช้ตารางไหนครับ Dashboard Aggregator ใช้ตาราง orders order_items กับ products

---

หน้า 20 · UC-09 แดชบอร์ดระดับบริหาร

พูด

UC-09 เป็นตัวเลขสำหรับผู้จัดการกับเจ้าของครับ ดูยอดขายย้อนหลังกับสินค้าขายดี คนอื่นเปิดไม่ได้

ใช้โมดูลเดียวกับ UC-08 ครับ ทดสอบด้วย TC-08 ร่วมกับ UC-08 เพราะอยู่ใต้ FR-07 เดียวกัน

อธิบายเส้น

เส้นแรก จาก UC-09 ไปหน้าจอ dashboard-exec คือ use case นี้ทำที่หน้าจอนี้ครับ

จากหน้าจอ dashboard-exec มีเส้นเดียว ไป GET /api/dashboard/exec คือหน้าจอนี้ส่งคำขอไปหลังบ้านผ่าน API ตัวนี้

GET /api/dashboard/exec วิ่งเข้าโมดูล Dashboard Aggregator

เส้นสุดท้ายไปช่องขวาสุด คือโมดูลไหนใช้ตารางไหนครับ Dashboard Aggregator ใช้ตาราง orders order_items กับ products

---

หน้า 21 · UC-10 ค้นหาประวัติการทำรายการ

พูด

UC-10 ย้อนดูได้ว่าใครทำอะไรเมื่อไรครับ เวลามีปัญหาจะได้ตามถูก ผู้จัดการค้นตามชื่อพนักงาน ประเภทการกระทำ หรือช่วงวันได้

ประวัติพวกนี้มาจากทุกส่วนของระบบ ทุกครั้งที่ขาย จ่ายของ หรือแก้สินค้า โมดูลนั้นจะจดลง audit_logs ในธุรกรรมเดียวกันเลย ถ้างานหลักยกเลิก ประวัติก็ยกเลิกด้วย ไม่มีประวัติปลอม

ทดสอบด้วย TC-11 กรองตามวันกับประเภท ได้ผลตรงทุกแถว และมีผู้ทำ เวลา กับ IP ครบ ผ่านครับ

อธิบายเส้น

เส้นแรก จาก UC-10 ไปหน้าจอ audit-logs คือ use case นี้ทำที่หน้าจอนี้ครับ

จากหน้าจอ audit-logs มีเส้นเดียว ไป GET /api/audit-logs คือหน้าจอนี้ส่งคำขอไปหลังบ้านผ่าน API ตัวนี้

GET /api/audit-logs วิ่งเข้าโมดูล Audit Log

เส้นสุดท้ายไปช่องขวาสุด คือโมดูลไหนใช้ตารางไหนครับ Audit Log ใช้ตาราง audit_logs กับ users

---

หน้า 22 · UC-11 การแจ้งเตือนในแอป

พูด

UC-11 ระบบเตือนเองเมื่อของใกล้หมดครับ หลังจ่ายของหรือปรับยอด Alert Engine เช็กว่ามีสินค้าไหนต่ำกว่าจุดสั่งซื้อเพิ่ม ถ้ามีก็สร้างแจ้งเตือนแล้วส่งไปที่จอทันที ผู้จัดการเปิดดูแล้วกดรับทราบ

ทดสอบสองข้อครับ TC-09 จ่ายจนต่ำกว่าจุดสั่งซื้อเพิ่ม แจ้งเตือนขึ้นเอง และไม่แจ้งซ้ำตอนที่ยังไม่อ่าน กับ TC-10 ปล่อยใบค้างในคิวนานเกินเวลา ระบบเตือนคิวค้างพร้อมบอกเลขใบกับเวลาที่รอ ผ่านทั้งสองข้อครับ

อธิบายเส้น

เส้นแรก จาก UC-11 ไปหน้าจอ notifications คือ use case นี้ทำที่หน้าจอนี้ครับ

จากหน้าจอ notifications มี 2 เส้น ไป /api/notifications (GET และ PATCH อ่านแล้ว) กับ WebSocket /ws (สำรองเป็น SSE) คือหน้าจอนี้ส่งคำขอไปหลังบ้านผ่าน API สองตัวนี้

/api/notifications (GET และ PATCH อ่านแล้ว) วิ่งเข้าโมดูล Alert Engine

เส้นโค้งทางขวาของช่อง Backend คือโมดูลเรียกกันเองครับ Alert Engine เรียก Queue Manager กับ Audit Log

เส้นสุดท้ายไปช่องขวาสุด คือโมดูลไหนใช้ตารางไหนครับ Alert Engine ใช้ตาราง notifications กับ products · Audit Log ใช้ตาราง audit_logs

---

หน้า 23 · ความครอบคลุมของงาน

พูด

ตอนนี้ทำครบทั้ง 11 use case แล้วครับ requirement ครอบคลุม 100% คือ FR ผ่าน 9 จาก 9 และ NFR ผ่าน 9 จาก 9 ตามตารางที่ 4.8

เทสต์ระดับ unit กับ integration ผ่าน 134 จาก 134 ส่วนเทสต์ระดับระบบ TC-01 ถึง TC-13 ผ่านครบ 13 ข้อครับ ลูกค้าของร้านไม่ได้ใช้ระบบเองนะครับ แต่ได้ผลผ่าน UC-03 กับ UC-05 คือไม่ต้องถือบิลเดินไปคลังแล้ว

ถ้าอาจารย์ถาม 100% วัดจากอะไร

นับจากตารางสืบย้อนในบทที่ 4 ครับ ทุก requirement มี use case รองรับ และมีกรณีทดสอบที่ผ่าน

ดูในเล่ม ตารางที่ 4.8

หน้า 24 และ 25

สองหน้านี้เพิ่มเองใน Canva ยังไม่มีข้อความบนสไลด์ จึงยังไม่มีบทพูด

คำถามที่ควรซ้อมเพิ่ม (ตามที่อาจารย์บอกว่าจะถาม)

1. **ปัญหาคืออะไร** ร้านเขียนบิลมือ ลูกค้าถือบิลเดินไปคลัง สต็อกไม่ตรงกับของจริง ไม่รู้ว่าของใกล้หมด (EV-01 ถึง EV-09)
2. **เก็บความต้องการมาแบบไหน** สังเกตหน้างาน สัมภาษณ์เจ้าของร้านกับพนักงาน จดเป็น EV
3. **requirement มีอะไรบ้าง แบ่งยังไง** 18 ข้อ FR 9 NFR 9 NFR แยกตามด้านคุณภาพ
4. **เอามาทำ use case ได้อะไร** 11 ตัว แบ่งตามคนทำและงานที่จบในตัวเอง (ตารางที่ 3.8)
5. **ออกแบบ C2 C3 ยังไง** C2 3 ชั้น 4 หน่วยติดตั้ง · C3 10 โมดูล 3 ส่วนสนับสนุน แยก Stock Deduction ออกมาเพื่อทดสอบพร้อมกัน